

Machine Learning II (BAI702)

IA2 Question Bank — Solved Answers

(Units 2, 3, 4)

IA2 Syllabus Coverage

Unit 2: Analytical Learning: Perfect Domain Theories: Explanation-Based Learning, Explanation-Based Learning of Search Control Knowledge, Inductive-Analytical Approaches to Learning.

Unit 3: Decision by Committee: Ensemble Learning: Boosting: AdaBoost, Stumping, Bagging: Subagging, Random Forests, Comparison with Boosting, Different ways to Combine Classifiers. Unsupervised Learning: The K-Means Algorithm: Dealing with Noise, The k-Means Neural Network, Normalisation, A Better Weight Update Rule, Using Competitive Learning for Clustering.

Unit 4: Unsupervised Learning: Vector Quantisation, The Self-Organising Feature Map, The SOM Algorithm.

Q1. Explain Explanation-Based Learning (EBL) with an example. How does it use perfect domain theories?

Answer:

Inductive learning methods generalize purely from observed training examples, and they perform poorly when only a small number of examples are available, because complex domains (such as a chess board with 32 pieces on 64 squares) would require thousands of examples before a correct hypothesis could be induced.

Explanation-Based Learning (EBL) solves this problem by allowing the learner to accept *explicit prior knowledge* as an additional input, along with the training data. This prior knowledge is called a **domain theory** B — a set of background rules/facts that can be used to *explain* why a given training example satisfies the target concept. By analysing this explanation, EBL determines exactly which features of the example are relevant to the target concept and which are irrelevant, and it uses this to generalize far more accurately than blind induction.

Formal definition. In inductive learning, the learner is given a hypothesis space H and training examples $D = \{(x_1, f(x_1)), \dots, (x_n, f(x_n))\}$, and must output some $h \in H$ consistent with D . In **analytical learning**, the learner is given the same H and D , but additionally a domain theory B of background knowledge. The desired output is a hypothesis $h \in H$ that is consistent with *both* D and B .

Perfect domain theories. A domain theory is called *correct* if every assertion in it is a truthful statement about the world, and *complete* with respect to a target concept if it can prove (cover) every positive instance of that concept in the instance space. When B is both correct and complete, it is called a **perfect domain theory**. EBL algorithms such as PROLOG-EBG assume a perfect domain theory: since a perfect theory can prove any positive example, it can equally well be used to predict future

classifications — so in this special case, EBL is really *reformulating* the knowledge already present in B into an efficient, directly usable rule.

Example — chess. Consider learning the target concept “*chessboard positions in which black will lose its queen within two moves.*” Figure 1 shows one positive example: a board on which white’s knight simultaneously attacks the black king and the black queen.

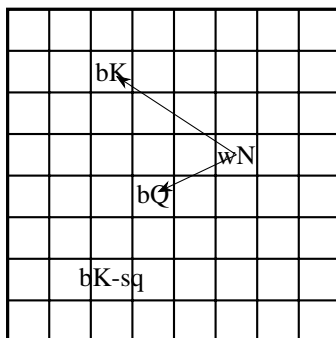


Figure 1: Positive example — white knight forks the black king and queen (schematic, after Fig. 11.1).

Given only this one example, a purely inductive learner cannot tell which of the many attributes present (piece colours, unrelated pawn positions, etc.) are actually relevant — it might just as easily hypothesize the irrelevant rule “boards in which four white pawns are still in their original squares.” EBL avoids this by *explaining* the example: it uses the domain theory (the legal moves of a knight, the rule that players alternate moves, and the rule that a king in check must move) to construct a proof that this configuration forces black to lose the queen. Only the facts actually used in this proof — namely that the knight simultaneously attacks the king and the queen — are kept, and everything else (colour of the pieces, position of irrelevant pawns) is dropped. This yields the correct, general rule: “*board positions in which the black king and queen are simultaneously attacked.*”

Thus EBL uses a small number of examples together with a perfect domain theory to produce a hypothesis whose generality is justified by explanation rather than by seeing many examples, which is exactly why it is not limited by the sample-complexity bounds of purely inductive learners.

Q2. Describe the working of the PROLOG-EBG algorithm in detail.

Answer:

PROLOG-EBG is an explanation-based generalization algorithm that assumes a **perfect domain theory** (correct and complete). It is a **sequential covering algorithm**: it processes the positive training examples incrementally, learning one Horn-clause rule at a time, until every positive example is covered by some learned rule.

Algorithm.

PROLOG-EBG(*TargetConcept*, *TrainingExamples*, *DomainTheory*)

1. $LearnedRules \leftarrow \{\}$
2. $Pos \leftarrow$ the positive examples from *TrainingExamples*
3. for each *PositiveExample* in Pos not covered by *LearnedRules*, do:

Explain: Construct an *Explanation* (a proof, in terms of the *DomainTheory*) that *PositiveExample* satisfies *TargetConcept*.

Analyze: Compute *SufficientConditions* — the most general set of features of *PositiveExample* that are sufficient, according to the *Explanation*, to satisfy *TargetConcept*.

Refine: $LearnedRules \leftarrow LearnedRules + NewHornClause$, where *NewHornClause* is of the form

$$TargetConcept \leftarrow SufficientConditions$$

4. Return *LearnedRules*

So for every new positive example not yet covered, the algorithm performs three steps: (1) it **explains** the example using the domain theory (a backward-chaining, Prolog-style proof that halts at the first valid proof found), (2) it **analyses** the explanation to find the most general conditions that justify it (this is done by computing the *weakest preimage* of the target concept with respect to the proof, see Q6), and (3) it **refines** the learned rule set by adding a new Horn clause built from those general conditions.

Example. For the target concept *SafeToStack*(*x*, *y*) (Table 1, Q5), the domain theory explains a positive example *SafeToStack*(*Obj1*, *Obj2*) via the proof: *Obj1* is *Lighter* than *Obj2* because *Weight*(*Obj1*) can be derived from its *Volume* and *Density*, and *Weight*(*Obj2*) from the default weight of an *Endtable*. Only the leaf facts mentioned in this proof (*Volume*, *Density* of *Obj1*; *Type* of *Obj2*) are kept, while irrelevant attributes such as *Color* or *Owner* are dropped, giving the general rule:

$$SafeToStack(x, y) \leftarrow Volume(x, 2) \wedge Density(x, 0.3) \wedge Type(y, Endtable)$$

Because PROLOG-EBG assumes the domain theory is correct and complete, the rules it produces are logically justified generalizations rather than merely statistically likely ones; however, if the domain theory is in fact incorrect or incomplete, the learned concept may also be incorrect.

Q3. Compare and contrast Inductive Learning vs Analytical Learning with examples.

Answer:

Inductive Learning. The learner is given a hypothesis space H and a set of training examples $D = \{(x_1, f(x_1)), \dots, (x_n, f(x_n))\}$, where $f(x_i)$ is the target value of instance x_i . The desired output is a hypothesis $h \in H$ that is *consistent with the training examples D* alone. Because it relies only on statistical regularities in D , it needs a large number of examples to generalize reliably and cannot distinguish relevant from irrelevant attributes unless this is reflected many times over in the data.

Analytical Learning. The learner is given the same H and D , but is additionally provided a **domain theory B** — background knowledge that can be used to *explain* observed training examples. The desired output is a hypothesis $h \in H$ consistent with *both D and B* . Because the domain theory tells the learner *why* an example is positive, analytical learning (e.g., EBL) can generalize correctly from very few examples — even a single example, as in the chess and *SafeToStack* cases.

Aspect	Inductive Learning	Analytical Learning
Input	H, D	H, D , and domain theory B
Output requirement	h consistent with D	h consistent with D and B
Data requirement	Needs many training examples	Can work from very few examples
Basis of generalization	Statistical regularity across examples	Logical explanation (proof) of why the example is positive
Relevant-feature detection	Must be inferred from repeated patterns	Directly identified from the explanation
Example	Learning <i>PlayTennis</i> from many days of weather data	Learning “black loses queen in two moves” from one annotated chess position using the rules of chess as B

Example contrast. To learn the chess target concept inductively, thousands of labelled board positions would be needed for the hypothesis to generalize correctly, because the chess board is fairly complex (32 pieces, 64 squares) and there are many superficially plausible but wrong hypotheses consistent with a handful of examples. Analytically, using the domain theory (legal moves of each piece, alternation of turns, capturing the king to win), a single example together with an explanation of *why* it is positive is enough to derive the correct general rule — “boards in which the black king and queen are simultaneously attacked.”

In short, inductive learning trades prior knowledge for data, while analytical learning trades data for prior knowledge; EBL is a method that specifically exploits a domain theory to reduce the hypothesis space that must be searched, thereby improving generalization accuracy from limited data.

Q4. Explain Explanation-Based Learning of Search Control Knowledge with the PRODIGY system.

Answer:

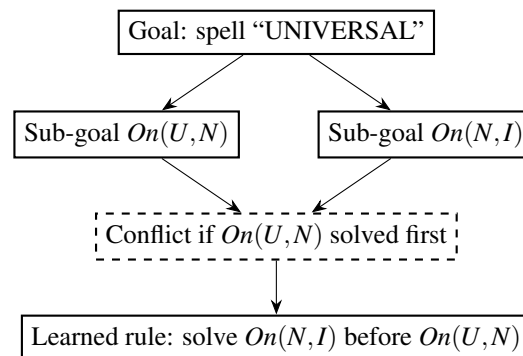
The practical applicability of PROLOG-EBG is restricted by its requirement that the domain theory be correct and complete. Even so, EBL can be applied inside **search programs** such as game-playing or planning systems, where it is used to learn *search-control knowledge* — rules that tell the search algorithm which move or sub-goal ordering to prefer, rather than rules that classify final states.

PRODIGY is one such system: it is a *domain-independent planning system*. It accepts a planning problem described in terms of a state space S and a set of operators O , and it must find a sequence of operators that leads from the initial state S_i to a state satisfying the goal G . To make this tractable, PRODIGY **decomposes** the overall problem into sub-problems (sub-goals), solves each sub-problem, and combines the sub-solutions into a single overall solution.

A key difficulty in this decomposition is that the sub-goals may **interact**: solving them in one order may be efficient, while solving them in a different order may force the planner to undo work it has already done (a *conflict*). PRODIGY uses EBL to learn control rules that avoid such conflicts.

Example — block stacking. Suppose the goal is to stack blocks so that they spell the word “universal.” PRODIGY decomposes this into sub-goals such as $On(U, N)$ and $On(N, I)$. If PRODIGY tries to solve $On(U, N)$ before $On(N, I)$, it discovers a conflict: it must later undo the $On(U, N)$ placement in order to achieve $On(N, I)$ (because, e.g., the robot arm can only hold one block at a time). PRODIGY *explains* this conflict to itself using its domain theory of the blocks-world operators, and from this explanation it learns the following general control rule:

IF one sub-goal to be solved is $On(x, y)$, and one sub-goal to be solved is $On(y, z)$
THEN solve the sub-goal $On(y, z)$ before $On(x, y)$



The net effect is that PRODIGY combines *domain-independent* knowledge about the general nature of sub-goal conflicts with *domain-specific* knowledge of the actual operators (e.g., that the robot can pick up only one block at a time) to learn useful, reusable, domain-specific planning/search-control rules such as the one above — this is a direct application of explanation-based learning to speeding up search rather than to classifying static instances.

Q5. With a neat diagram, explain the $\text{SafeToStack}(x,y)$ example used in explanation-based learning.

Answer:

The $\text{SafeToStack}(x,y)$ problem is a classic illustration of analytical learning. It is defined as follows:

- **Instance space X :** pairs of objects described by predicates *Type*, *Color*, *Volume*, *Owner*, *Material*, *Density*, *On*.
- **Hypothesis space H :** sets of Horn-clause rules whose head is a literal containing the target predicate *SafeToStack*, and whose body is a conjunction of literals over the above predicates plus *LessThan*, *Equal*, *GreaterThan*, and arithmetic functions *plus*, *minus*, *times*.
- **Target concept:** $\text{SafeToStack}(x,y)$.
- **Training example (positive):** $\text{SafeToStack}(\text{Obj1}, \text{Obj2})$, with

$\text{On}(\text{Obj1}, \text{Obj2}), \text{Type}(\text{Obj1}, \text{Box}), \text{Type}(\text{Obj2}, \text{Endtable}),$
 $\text{Color}(\text{Obj1}, \text{Red}), \text{Color}(\text{Obj2}, \text{Blue}), \text{Volume}(\text{Obj1}, 2),$
 $\text{Owner}(\text{Obj1}, \text{Fred}), \text{Owner}(\text{Obj2}, \text{Louise}), \text{Density}(\text{Obj1}, 0.3),$
 $\text{Material}(\text{Obj1}, \text{Cardboard}), \text{Material}(\text{Obj2}, \text{Wood})$

- **Domain theory B (partial):**

$\text{SafeToStack}(x,y) \leftarrow \neg \text{Fragile}(y)$
 $\text{SafeToStack}(x,y) \leftarrow \text{Lighter}(x,y)$
 $\text{Lighter}(x,y) \leftarrow \text{Weight}(x,w_x) \wedge \text{Weight}(y,w_y) \wedge \text{LessThan}(w_x,w_y)$
 $\text{Weight}(x,w) \leftarrow \text{Volume}(x,v) \wedge \text{Density}(x,d) \wedge \text{Equal}(w,v \times d)$
 $\text{Weight}(x,5) \leftarrow \text{Type}(x, \text{Endtable})$
 $\text{Fragile}(x) \leftarrow \text{Material}(x, \text{Glass})$

Explanation. The domain theory proves $\text{SafeToStack}(\text{Obj1}, \text{Obj2})$ because *Obj1* is *Lighter* than *Obj2*: $\text{Weight}(\text{Obj1}) = \text{Volume}(\text{Obj1}) \times \text{Density}(\text{Obj1}) = 2 \times 0.3 = 0.6$, and $\text{Weight}(\text{Obj2}) = 5$ from the default weight of an *Endtable*; since $0.6 < 5$, *Obj1* is lighter, hence safe to stack.

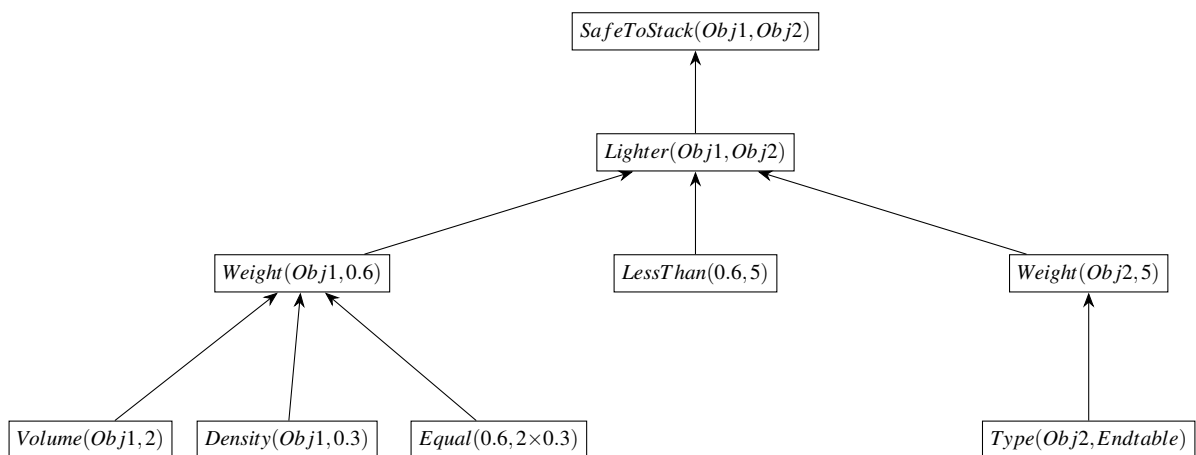


Figure 2: Explanation (proof tree) of the training example $\text{SafeToStack}(\text{Obj1}, \text{Obj2})$ (after Fig. 2, based on Table 1).

The explanation mentions only the shaded leaf attributes (*Volume*, *Density* of *Obj1*; *Type* of *Obj2*) — it never refers to *Owner* or *Color*. Collecting only these leaf features and replacing *Obj1*, *Obj2* with

variables x, y gives the justified general rule:

$$SafeToStack(x, y) \leftarrow Volume(x, 2) \wedge Density(x, 0.3) \wedge Type(y, Endtable)$$

(three conditions: x 's volume is 2, x 's density is 0.3, and y is an endtable)

This is a genuine generalization: it omits irrelevant properties (Color, Owner) while retaining exactly the attributes the domain theory shows to be relevant to the target concept.

Q6. Explain the process of weakest preimage generation in PROLOG-EBG.

Answer:

Once an explanation (proof tree) has been constructed for a positive training example, PROLOG-EBG must decide how general a rule can be justified by that same proof structure. The technique used is called computing the **weakest preimage** of the target concept with respect to the explanation.

Definition. The weakest preimage of a conclusion C with respect to a proof P is the *most general* set of initial assertions A such that A entails C according to P . In other words, instead of only keeping the specific constant values that appeared in the one training example, PROLOG-EBG works backwards (*regresses*) through each step of the proof tree, replacing the specific values with the most general conditions that would still make that step of the proof valid.

Process (regression). PROLOG-EBG iterates through the explanation from the root (the target concept) down to the leaves: at each step it computes the weakest preimage of the current expression with respect to that step of the proof, then uses that result as the new expression whose weakest preimage is computed at the next step, and so on, until it has processed every step in the explanation. The final result is the weakest (most general) condition on the target concept that this particular explanation still justifies.

Example. For $\text{SafeToStack}(\text{Obj1}, \text{Obj2})$, if we simply substituted variables for the specific constants in Figure 2's leaves, we would get the rule

$$\text{SafeToStack}(x, y) \leftarrow \text{Volume}(x, 2) \wedge \text{Density}(x, 0.3) \wedge \text{Type}(y, \text{Endtable})$$

which still requires the *exact* values $\text{Volume} = 2$ and $\text{Density} = 0.3$. But by computing the weakest preimage — regressing the proof so that only the *relationship* between the values (not their specific magnitudes) is retained — PROLOG-EBG derives the more general rule:

$$\text{SafeToStack}(x, y) \leftarrow \text{Volume}(x, v_x) \wedge \text{Density}(x, d_x) \wedge \text{Equal}(w_x, v_x \times d_x) \wedge \text{LessThan}(w_x, 5) \wedge \text{Type}(y, \text{Endtable})$$

(in words: x 's weight, worked out from its volume and density, must be less than the endtable's weight of 5)

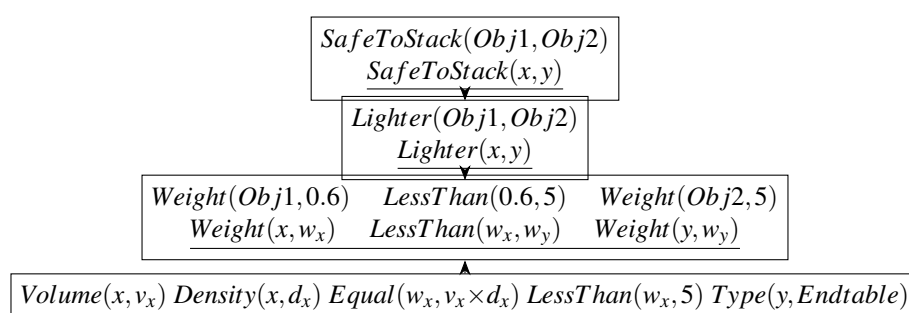


Figure 3: Weakest preimage computed by regression — underlined expressions are the generalized (variablized) form at each level (after the weakest-preimage figure).

This more general rule no longer needs the specific values 2 and 0.3; it instead states the general *constraint* that the weight computed from *Volume* and *Density* must be less than 5. Regression therefore lets PROLOG-EBG extract the most general rule that the given proof can logically support, rather than a rule tied to the constants of one specific example.

Q7. Discuss Inductive-Analytical approaches and how they combine the strengths of both learning methods.

Answer:

Pure inductive learning uses only a hypothesis space H and training data D ; it can suffer from poor generalisation when D is small, because many hypotheses in H remain consistent with a small D and there is no principled way to prefer one over another. Pure analytical learning (EBL) uses H , D , and a domain theory B , and can generalise from very few examples — but this comes at the cost of needing B to be correct and complete; when B is imperfect (incomplete or slightly incorrect, as most real domain theories are), EBL alone can fail or learn an incorrect concept.

Inductive-analytical approaches sit between these two extremes: the learner is again given H , D , and B , but B is allowed to be imperfect. The goal is a hypothesis $h \in H$ that fits the training data *well* while also being as consistent as possible with the (possibly imperfect) domain theory, so that:

- where B gives a clear explanation, that explanation is used to bias the search of H toward relevant features (as in EBL) — reducing the number of examples needed and improving the generality of the resulting rule;
- where B is silent or wrong on a particular example, ordinary inductive search over D is still used to fill the gap, so the learner is not solely dependent on the theory being complete.

Why this combines the strengths of both.

- From **analytical learning** it inherits the ability to use prior knowledge to distinguish relevant example attributes from irrelevant ones (as PROLOG-EBG does via explanation and weakest-preimage regression), which allows it to avoid the sample-complexity bounds that limit purely inductive learners — i.e., it needs fewer examples to reach a correct generalisation.
- From **inductive learning** it inherits robustness to an imperfect domain theory: because the final hypothesis is still checked and refined against the observed training examples D , errors or gaps in B do not automatically propagate into the learned concept the way they would in a system (like PROLOG-EBG) that assumes B is perfect.

Illustration using the *SafeToStack* setting. If the domain theory only partially explains why some object pairs are safe to stack (e.g., it knows about *Lighter* and *Fragile* but has no rule at all for certain materials), a purely analytical learner might be unable to explain some positive examples and would fail to learn anything from them. An inductive-analytical learner can still use the partial explanation where available (to correctly ignore irrelevant attributes such as *Color* or *Owner*, exactly as PROLOG-EBG does) while falling back on ordinary inductive generalisation from the remaining, unexplained examples — thus combining the *sample efficiency and relevance-detection* of EBL with the *robustness to imperfect prior knowledge* of purely inductive learning.

In summary, inductive-analytical learning is motivated by the practical observation that real domain theories are rarely perfect: it retains the mechanism of EBL (explain, analyse relevant features, refine into general rules) as a bias on the hypothesis search, while still allowing the training data to inductively correct or supplement the theory where it is incomplete.

Q8. Explain the working of the AdaBoost algorithm with an example.

Answer:

AdaBoost (Adaptive Boosting) is the most widely used **boosting** algorithm. Boosting trains a sequence of weak/base classifiers, where each new classifier focuses on correcting the mistakes of the previous ones; the final prediction is a weighted majority vote (classification) or weighted sum (regression) of all the classifiers.

Idea. AdaBoost's innovation is to give each training datapoint a **weight** reflecting how difficult it has been for previous classifiers to get it correct. These weights are supplied to the classifier as part of its input at training time. Initially all weights are equal, $w_n = 1/N$ for each of the N datapoints. At each iteration:

1. a new classifier is trained on the (weighted) training set, producing hypothesis $h_t(x_n)$;
2. the weighted training error is computed, $\epsilon_t = \sum_{n=1}^N w_n^{(t)} I(y_n \neq h_t(x_n))$, where $I(\cdot)$ is 1 if the condition is true and 0 otherwise;
3. a classifier-confidence value is set, $\alpha_t = \log\left(\frac{1 - \epsilon_t}{\epsilon_t}\right)$ (equivalently, weights of incorrect points are multiplied by $\alpha = (1 - \epsilon)/\epsilon$);
4. the weights are updated: $w_n^{(t+1)} = w_n^{(t)} \exp(\alpha_t I(y_n \neq h_t(x_n))) / Z_t$, where Z_t is a normalising constant so the weights sum to 1.

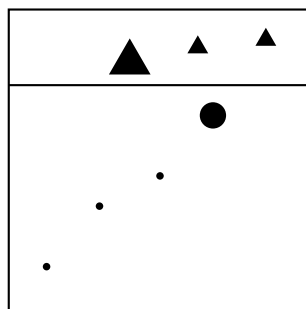
Weights of *correctly* classified points are left unchanged and then renormalised, which effectively *reduces* their relative importance, while incorrectly classified points get relatively larger weight, forcing the next classifier to pay more attention to them. Training stops after T iterations, or when all points are classified correctly, or when one point accumulates more than half the total weight. The final output is

$$f(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right)$$

i.e. a weighted vote of all the weak classifiers, weighted by how accurate each one was.

Explaining “easy in plain terms”: ϵ_t is simply the fraction of (weighted) training points the t -th classifier got wrong; α_t is large when ϵ_t is small (an accurate classifier gets a bigger vote) and close to 0 when ϵ_t is near 0.5 (a nearly-random classifier gets almost no vote); the weight-update formula simply *multiplies up* the weight of every point the current classifier misclassified (making it e^{α_t} times as important) and leaves correctly classified points alone, then rescales everything so the weights still sum to 1.

Example. A very simple base classifier that can only fit a single horizontal or vertical line was boosted on a 2-D dataset where the top-right region is one class and the rest is another (with a couple of points deliberately mislabelled as noise). No single horizontal/vertical line can separate this data. Figure 4 shows the effect of boosting: as training proceeds, misclassified points are given larger weight (represented by larger point size), so later classifiers focus more on them.



horizontal decision line, with one boundary point up-weighted (large marker)

Figure 4: Boosting increases the weight (marker size) of misclassified/borderline points across iterations, making the classifiers pay more attention to them (after Fig. 13.2).

On an independent test set, this simple boosted ensemble of horizontal/vertical-line classifiers achieved only one misclassified point — one that happened to lie close to a noisy training point — despite each individual weak classifier being far weaker on its own. This demonstrates AdaBoost's advantage: reducing both bias and variance and often achieving state-of-the-art performance, although it is more prone to overfitting than bagging if the number of iterations is not controlled (i.e., not regularised).

Q9. Describe the Random Forest algorithm with steps. Why is it less prone to overfitting?

Answer:

A **Random Forest** is an ensemble of decision trees, based on the idea that if one tree is good, many sufficiently different trees (a forest) should be better. It creates diversity between the trees using *two* sources of randomness:

1. **Bagging:** each tree is trained on a different *bootstrap sample* (a sample of the same size as the original dataset, drawn *with replacement*) of the training data.
2. **Random feature subsets:** at each node of each tree, instead of considering all features, only a random subset of m features is considered, and the best split (by information gain or Gini impurity) is chosen only from that subset. A common default is $m = \sqrt{(\text{total number of features})}$.

The Basic Random Forest Training Algorithm.

For each of N trees:

- create a new bootstrap sample of the training set;
- use this bootstrap sample to train a decision tree;
- at each node of the tree, randomly select m features, and compute the information gain (or Gini impurity) only on that set of features, selecting the optimal one;
- repeat until the tree is complete.

Once all N trees are trained, the forest's output is the **majority vote** across trees for classification, or the **mean** response for regression.

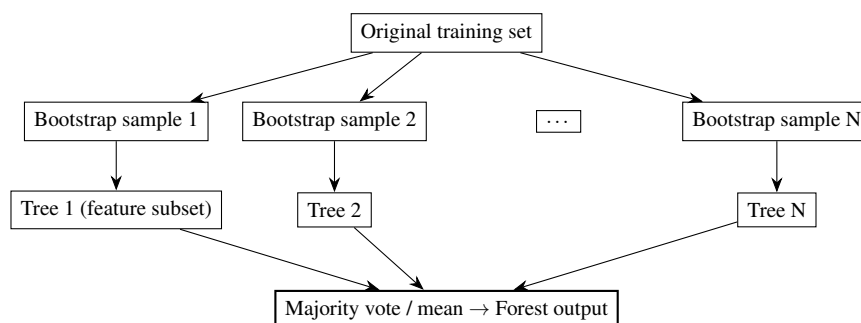


Figure 5: Random Forest training — bagging plus per-node random feature subsets.

Example results (car evaluation dataset, 1728 examples, UCI repository). Comparing a single decision tree, bagging, and a random forest of 50 trees (each depth ≤ 5 , trained on 100-sample bootstraps):

Method	Overall accuracy	Accuracy on “good/very good” cars
Single Tree	89.93%	46.15%
Bagger	78.47%	0.00%
Random Forest	91.78%	51.28%

The random forest is the most accurate of the three.

Why it is less prone to overfitting. A single deep decision tree can fit the noise in the training set very closely (low bias, high variance). By averaging (or majority-voting) over many trees that are each trained on a different bootstrap sample and a different random subset of features at every node, the errors

of individual trees — which tend to be uncorrelated because the trees are trained on different data/feature subsets — cancel out in the aggregate, so the variance of the ensemble is much lower than the variance of any single tree, without a large increase in bias. Additionally, restricting each split to a random subset of m features stops a few dominant features from making all the trees identical, further increasing the diversity (decorrelation) between trees, which is precisely what keeps the ensemble from overfitting to the particular training data.

Q10. Discuss different methods of combining classifiers with suitable examples.

Answer:

Two questions matter for any ensemble method: *how the different classifiers are made to differ*, and *how their outputs are combined*. This answer focuses on the second question — ways of combining the outputs of an already-trained set of classifiers.

1. Majority voting. Each classifier casts one vote for its predicted class, and the class with the most votes wins. This is what **bagging** uses. Even majority voting is not entirely simple: some systems require *all* classifiers to agree, or more than half to agree, before producing an output at all (to avoid answering on “contentious”, probably-difficult points), while the usual meaning is simply to take the most common output.

If there are T independent classifiers, each with success rate $p > 0.5$, the probability that the majority vote is correct is a binomial sum:

$$\sum_{k=T/2+1}^T \binom{T}{k} p^k (1-p)^{T-k}$$

This sum approaches 1 as $T \rightarrow \infty$: even if each classifier is correct only slightly more than half the time, using enough classifiers (e.g., $T = 100$) makes the ensemble’s overall accuracy very close to 1. This binomial argument is the core mathematical justification for ensemble methods.

2. Weighted voting. This is what **boosting** (e.g., AdaBoost) uses: rather than every classifier getting one equal vote, each classifier’s vote is weighted by $\alpha_t = \log((1 - \epsilon_t)/\epsilon_t)$, its own measure of accuracy, so more accurate classifiers have proportionally more influence on the final decision, $f(x) = \text{sign}(\sum_t \alpha_t h_t(x))$.

3. Mean / median (for regression). For regression problems, rather than a majority vote, it is common to take the **mean** of the individual outputs. Since the mean is heavily affected by outliers, the **median** is a more robust alternative — using the median in place of the mean gives the *bragging* algorithm (“robust bagging”).

4. Mixture of Experts. Here, rather than fixing how to combine classifiers in advance, the combination itself is *learned*. Each expert (classifier) makes an assessment of the input, and each assessment is weighted by a **gate**, which computes a weight w from the current input; the weighted outputs are propagated up a hierarchy of gates.

$$o_i(\mathbf{x}, \mathbf{w}_i) = \frac{1}{1 + \exp(-\mathbf{w}_i \cdot \mathbf{x})}, \quad g_i(\mathbf{x}, \mathbf{v}_i) = \frac{\exp(\mathbf{v}_i \mathbf{x})}{\sum_l \exp(\mathbf{v}_l \mathbf{x})}, \quad \text{combined output} = \sum_k o_j g_j$$

Here o_i is each expert’s own probability-style output (a logistic function of its weights and the input), g_i is the softmax gate value that decides how much to trust that expert for this particular input, and the final combined value is simply the gate-weighted sum of expert outputs, propagated up through however many levels of gates the hierarchy has.

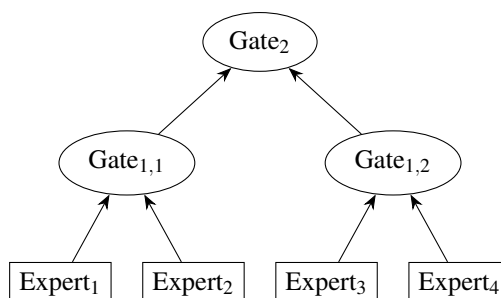


Figure 6: Hierarchical Mixture of Experts — experts with gating networks that decide which experts to trust for a given input (after Fig. 13.5).

In summary: bagging uses simple majority voting, boosting uses weighted (confidence-based) voting, regression ensembles use the mean or the more robust median, and mixture-of-experts learns input-dependent weights via gating networks rather than fixing the combination rule in advance.

Q11. Describe the importance of normalisation in clustering with an example.

Answer:

In the k-means **neural network** formulation of clustering, cluster centres are represented as the weight vectors of a layer of *competitive* neurons, and each neuron's activation is computed as

$$h_i = \sum_j w_{ij} x_j = \mathbf{W}_i^T \cdot \mathbf{x}$$

i.e., the inner (dot) product between the neuron's weight vector and the input. The inner product equals $\|\mathbf{W}_i\| \|\mathbf{x}\| \cos \theta$, where θ is the angle between the two vectors. This means the size of the activation depends on *both* the angle between the vectors *and* their magnitudes ($\|\mathbf{W}_i\|$ and $\|\mathbf{x}\|$) — not on angle (closeness in direction) alone.

Why this is a problem. Suppose all neurons have small weights (say less than 1) except one neuron whose weights are all 10. For an input vector $(0.2, 0.2, -0.1)$ that is an *exact match* for one of the small-weight neurons, that neuron's activation is $0.2 \times 0.2 + 0.2 \times 0.2 + (-0.1) \times (-0.1) = 0.09$. But the neuron with large weights (even though it is a poor match in direction) produces activation $10 \times 0.2 + 10 \times 0.2 + 10 \times (-0.1) = 3$, which is much larger. The large-weight neuron would incorrectly “win” simply because its weight vector is longer, not because it is actually closer to the input.

The fix — normalisation. To compare neurons fairly by direction alone, every weight vector is normalised so that its Euclidean distance from the origin is exactly 1 — i.e., every neuron is positioned on the **unit hypersphere** (a circle in 2D, a sphere in 3D, as in Figure 7). If, in addition, the input vectors are also normalised to unit length, then $h_i = \mathbf{W}_i^T \cdot \mathbf{x} = \cos \theta$, so the activation depends *only* on the angle between the weight vector and the input — the winning neuron is now genuinely the one whose direction is closest to the input.

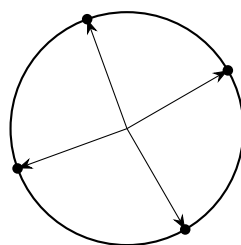


Figure 7: Weight vectors normalised onto the unit circle (unit hypersphere in higher dimensions), so only direction — not magnitude — determines the winning neuron (after Fig. 14.4).

In NumPy, this normalisation is computed by dividing each row of the data matrix by its own Euclidean norm (row-wise), which requires transposing before and after the division since NumPy's default broadcasting operates column-wise:

```
normalisers = sqrt(sum(data**2, axis=1))
data = transpose(transpose(data)/normalisers)
```

Consequence for the weight update rule. The basic weight update rule $\Delta w_{ij} = \eta h_i x_j$ (used for the winning neuron in the un-normalised k-means neural network) lets the weights grow without bound, moving them off the unit hypersphere again. This motivates the improved update rule (Q11 continuation, Q12) $\Delta w_{ij} = \eta (x_j - w_{ij})$, which moves the weight vector *directly towards* the current input and keeps normalisation meaningful over successive updates. In short, normalisation is essential in competitive-learning clustering because, without it, neurons can win purely by having larger weight magnitudes rather than by being genuinely closer in direction to the input, which would make the clustering meaningless.

Q12. Explain how k-means can be implemented using a neural network model.

Answer:

The standard k-means algorithm repeatedly (i) assigns each datapoint to its nearest cluster centre and (ii) moves each centre to the mean of the points assigned to it. This computation — “which cluster centre is each input closest to?” — can equally be implemented using a simple **competitive neural network**.

Architecture. The network has just *one layer of neurons* plus the input nodes (no hidden layers, no bias node). The input layer performs no computation, as usual. The second layer consists of k **competitive neurons** — neurons that “compete” to fire, with only the single most active neuron actually succeeding (winner-take-all); each cluster centre corresponds to exactly one competitive neuron, represented by its weight vector.

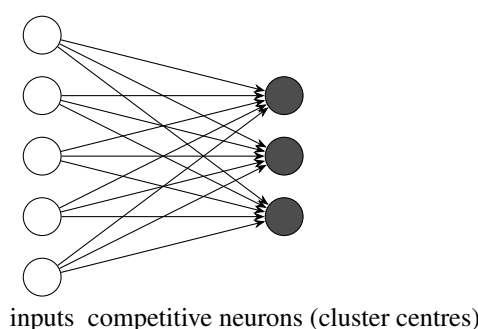


Figure 8: A single-layer competitive network implementing the k-means solution — inputs fully connect to k competitive neurons (after Fig. 14.3).

Activation. Each neuron uses a linear transfer function, computing its activation as the dot product of its weight vector and the input:

$$h_i = \sum_j w_{ij} x_j$$

Provided both weights and inputs are **normalised** to unit length (see Q11), this activation equals $\cos \theta$, i.e. it measures how close in *direction* the input is to that neuron’s weight vector — larger activation means the input and the cluster centre are closer together. The **winning neuron** is the one with the highest activation h , exactly playing the role of “nearest cluster centre” in ordinary k-means.

The On-Line k-Means Algorithm.

Initialisation: choose k (number of output nodes); initialise weights to small random values.

Learning: normalise the data onto the unit sphere; repeat: for each datapoint, compute the activation of every node, pick the winner as the node with highest activation, and update its weights using

$$\Delta w_{ij} = \eta (x_j - w_{ij})$$

until the number of iterations exceeds a threshold.

Usage: for each test point, compute the activations of all nodes and pick the winner as the cluster assignment.

Only the *winning* neuron’s weights are updated at each step (unlike batch k-means, which recomputes the mean of an entire cluster at once); the update rule $\Delta w_{ij} = \eta (x_j - w_{ij})$ moves the weight vector a small step *directly towards* the current input, so that, over many presentations, the winning neuron’s

weights converge towards the mean of the inputs that repeatedly select it — reproducing, in an online (one-input-at-a-time) fashion, exactly the behaviour of batch k-means, $\mu_j = \frac{1}{N_j} \sum_i \mathbf{x}_i$.

Using the trained network for clustering. Once trained, deciding which cluster a datapoint belongs to is simple: present it to the network and see which neuron fires (has the highest activation). If class labels are needed, care must be taken to match the arbitrary order of the output nodes to the target classes; alternatively, a Perceptron can be trained *on top of* the competitive layer in a supervised manner, so that the perceptron learns the correct output-to-class-label mapping regardless of node ordering.

Q13. Explain Bagging in detail with an example. How does it improve model stability?

Answer:

Bagging (*Bootstrap Aggregating*) is the simplest method of combining classifiers into an ensemble. It works by making the different classifiers see *different samples* of the data, rather than changing the importance of datapoints (as boosting does) or the features considered (as random forests do).

Bootstrap sampling. A bootstrap sample is obtained by sampling from the original dataset *with replacement*, so that the new sample is the same size as the original, but some original points may appear multiple times while others may not appear at all. B such bootstrap samples are generated, where B is at least 50 and can be in the thousands.

```
# Compute bootstrap samples
samplePoints = np.random.randint(0,nPoints,(nPoints,nSamples))
classifiers = []
for i in range(nSamples):
    sample, sampleTarget = [], []
    for j in range(nPoints):
        sample.append(data[samplePoints[j,i]])
        sampleTarget.append(targets[samplePoints[j,i]])
    # Train classifiers
    classifiers.append(self.tree.make_tree(sample, sampleTarget, features))
```

A separate classifier (e.g., a decision tree) is trained on each bootstrap sample. Although sampling with replacement from a dataset of the same size might seem wasteful, its real benefit is that it produces B classifiers that each perform *slightly differently* from one another — exactly what is needed for an ensemble method to work.

Combining outputs. Given the B trained classifiers, bagging combines their predictions by simple **majority voting** for classification, or by **averaging** for regression.

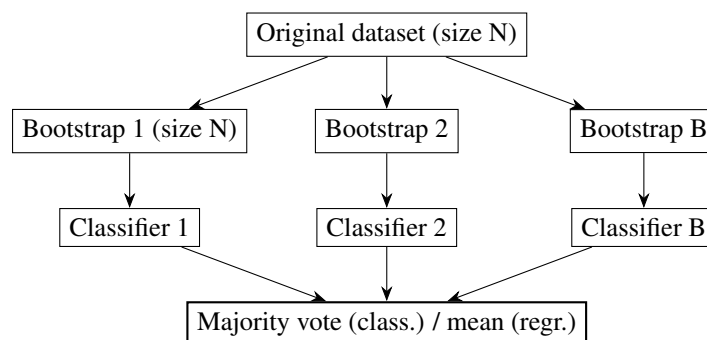


Figure 9: Bagging — B bootstrap samples train B classifiers, combined by majority voting / averaging.

Subagging (Subsample Aggregating). A variant of bagging: instead of bootstrap samples (with replacement, same size N), subagging draws **subsamples of size $m < N$ without replacement**.

Steps in Subagging:

1. Take the dataset of size N .
2. Randomly choose a subsample of size $m < N$ (without replacement).
3. Train a base learner (e.g., a decision tree) on this subsample.

4. Repeat many times to create multiple base learners.
5. Aggregate predictions: majority voting for classification, average for regression.

Example (car evaluation dataset, 1728 examples). A single decision tree (max depth 5) achieved 89.93% overall accuracy but only 46.15% accuracy on the “good/very good” minority class. A Bagger built from 50 trees (each trained on 100-sample bootstraps) achieved 78.47% overall accuracy but **0%** accuracy on the minority class in this particular run — illustrating that bagging’s benefit is not guaranteed on every dataset/class split, though it generally does reduce variance.

How bagging improves stability. A single classifier (especially a deep decision tree) trained on one dataset can be highly sensitive to the exact composition of that dataset — a small change in the training data can produce a very different tree (high variance/instability). Because each bootstrap sample is a slightly different resampling of the data, the B trained classifiers make *different errors* on different points; when their predictions are combined by majority vote or averaging, uncorrelated errors tend to cancel out, so the ensemble’s overall prediction is far more stable (lower variance) from one training run, or one slightly different dataset, to the next than any single classifier’s prediction would be — while the average bias of the ensemble stays close to the bias of the individual base learners.

Q14. Discuss the role of competitive neurons in clustering using k-means neural networks.

Answer:

In the k-means neural network, each of the k output neurons directly represents one cluster centre, stored as that neuron's weight vector. These output neurons are called **competitive neurons** because, for any given input, they *compete* to fire, and only the single neuron with the highest activation is declared the winner — this behaviour is also called **winner-take-all**.

Role in the forward pass (assignment step). Each neuron computes a linear activation $h_i = \sum_j w_{ij}x_j$. When both weights and inputs are normalised onto the unit hypersphere (Q11), this activation is $\cos \theta$, the cosine of the angle between the neuron's weight vector and the input vector. The neuron whose weight vector is closest *in direction* to the current input therefore produces the highest activation and wins the competition. This exactly replicates the “assign each datapoint to its nearest cluster centre” step of ordinary k-means, i.e.

$$n_b = \min_j d(\mathbf{x}, \mu_j)$$

except computed here via maximum activation among competitive neurons rather than via an explicit distance calculation.

Role in the learning (update) step. Only the **winning** neuron has its weights updated after each input is presented; all other (losing) neurons are left unchanged. This is what makes the neurons “compete” rather than cooperate: each neuron specialises to represent the region of input space for which it wins most often, and only that neuron adjusts its position (weights) to better match the inputs that select it. The improved update rule used is

$$\Delta w_{ij} = \eta (x_j - w_{ij})$$

which nudges the winning neuron's weight vector a small step *towards* the current input. Repeating this over many inputs causes each competitive neuron's weight vector to converge towards the **mean** of the inputs that consistently activate it most strongly — exactly reproducing the cluster-centre update $\mu_j = \frac{1}{N_j} \sum_i \mathbf{x}_i$ of batch k-means, but done incrementally, one input at a time, rather than in one big batch recomputation.

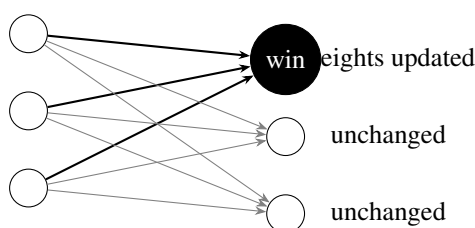


Figure 10: In competitive learning, only the winning neuron's weights are updated after each input; losing neurons are left unchanged.

Role in cluster assignment (usage phase). After training, assigning any test point to a cluster is simple: present it to the network and observe which competitive neuron fires (has the highest activation) — that neuron's index *is* the cluster label. If class labels are required, either the arbitrary neuron ordering must be matched carefully to the target classes, or a Perceptron layer can be trained on top of the competitive layer to learn this mapping supervised, which also allows using more clusters than there are target classes without having to hand-match them.

In summary, competitive neurons play a dual role: at run time they act as the “nearest cluster centre” decision mechanism, and during training the winner-take-all rule ensures that only the most relevant

neuron is adjusted for each input, so that different neurons naturally specialise into representing different clusters of the input space.

Q15. Explain how clustering can be used for noise reduction in datasets.

Answer:

Clustering methods such as k-means naturally support noise reduction because they replace each input datapoint by a single **prototype** (the cluster centre it belongs to). If the clusters are chosen correctly, this act of replacement effectively removes noise: instead of using the raw, possibly-corrupted measurement, the cleaner cluster-centre value is used in its place.

The mean is not robust. The mean average is the natural choice for computing the “middle” of a cluster’s points (it is the point halfway along the line between two points, generalised to many points, and is the correct centre in flat/Euclidean space). However, the mean is known to be very sensitive to **outliers**. For example, the mean of (1, 2, 1, 2, 100) is 21.2 — a single extreme (noisy) value drags the mean far away from where the “typical” data actually lies. If a cluster centre is computed using the mean and one of its member points is an extreme noisy reading, the resulting centre is pulled toward that noisy point, which defeats the purpose of noise removal.

The fix — use the median. The median is a **robust statistic**: it is not affected by outliers in the same way (the median of (1, 2, 1, 2, 100) is 2, correctly reflecting the bulk of the data). The only change needed to the k-means algorithm to make it robust to noise is to replace the computation of the mean, in the cluster-centre update step

$$\mu_j = \frac{1}{N_j} \sum_{i=1}^{N_j} \mathbf{x}_i$$

with the computation of the **median** of the points in that cluster. This is computationally more expensive than computing a mean, but it removes noise from the cluster centres far more effectively.

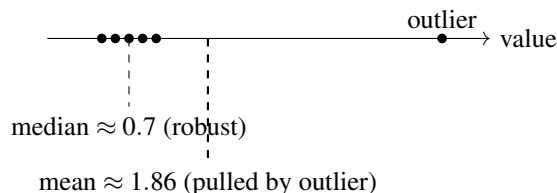


Figure 11: An outlier drags the mean away from where most of the data lies; the median stays close to the bulk of the (clean) data.

Effect on the dataset overall. Once training has converged, every original (possibly noisy) datapoint $\mathbf{x}_i$ is represented, for downstream use, by the centre μ_j of the cluster it was assigned to, rather than by its own raw value. Because the median-based centre is not distorted by any single noisy point in the cluster, this substitution effectively *filters out* occasional corrupted or plain-wrong readings while preserving the genuine underlying structure represented by the bulk of the points in each cluster — this is the same underlying mechanism (replacing an input by its nearest prototype) that is exploited for **data compression** in vector quantisation (Q16), the only difference being the motivation: noise removal versus reducing the amount of data transmitted.

Q16. Explain Vector Quantisation.

Answer:

Vector Quantisation (VQ) is an application of competitive learning / clustering to **data compression**, used both for storing data efficiently and for transmitting speech and image data. It is closely related to noise reduction (Q15): both replace the current input by the cluster centre (prototype) it belongs to — for noise reduction this cleans up a noisy input, while for compression it reduces the amount of data that needs to be sent or stored.

Motivating scenario. Suppose data must be transmitted, but every bit sent has a cost, so the amount of data sent should be minimised. If the data contains many repeated (or similar) datapoints, sender and receiver can agree in advance on a **codebook** of prototype vectors. Then, instead of transmitting a full datapoint, the sender transmits only the (short) *index* of the matching prototype in the codebook; the receiver looks up that index in the shared codebook to reconstruct the data. The code can be made even more efficient by giving shorter indices to more common prototypes (an information-theoretic idea used by every practical sound/image compression algorithm).

Lossy compression. A codebook cannot possibly contain every conceivable datapoint. When a datapoint to be sent is not exactly in the codebook, the sender instead transmits the index of the prototype vector *closest* to it, and the receiver's reconstruction will not be exactly identical to the original — this is precisely vector quantisation, and it is the reason this style of compression is **lossy**.

Voronoi tessellation. Geometrically, each prototype vector effectively “owns” a region of the input space consisting of every point closer to it than to any other prototype; this region is called the prototype's **Voronoi set**, and the collection of all such regions is the **Voronoi tessellation** of the space. Any datapoint that lies within a given cell is represented (reconstructed) by that cell's prototype vector (the dot at its centre).

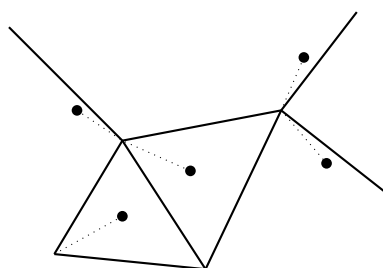


Figure 12: Voronoi tessellation performing vector quantisation. Each dot is a prototype (codebook) vector; any datapoint inside its cell is represented by that dot (after Fig. 14.5).

Relation to k-means / competitive learning. The prototype vectors used in VQ are exactly the cluster centres found by an algorithm such as k-means or the competitive-learning neural network: training on a representative dataset positions the prototypes so that they represent the input distribution well, and thereafter any new/incoming datapoint can simply be replaced (for storage/transmission purposes) by the index of its nearest prototype. This is why VQ is presented as a direct application of the same competitive-learning machinery already used for clustering and noise reduction, applied here with the goal of reducing the amount of data communicated rather than cleaning it.

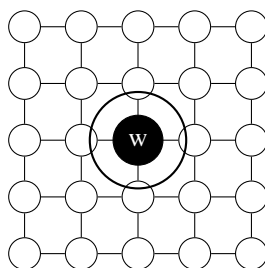
Q17. Discuss self-organisation in the context of SOM with an example.

Answer:

The name “Self-Organising Map” (SOM) refers to a particularly striking property of the network: a **global ordering** of the neurons in the network emerges even though the interactions used to train the network are entirely **local**. Neurons that are very far apart on the map never interact directly with each other during training, and yet the network as a whole ends up globally ordered, so that nearby neurons represent similar inputs and distant neurons represent dissimilar inputs. This phenomenon — global order arising purely from local interactions — is called **self-organisation**, and it is an instance of the more general science of complexity, appearing in many natural systems.

Analogy — flocking birds. A classic illustration of self-organisation outside neural networks is a flock of birds flying in formation. No bird can know the position of every other bird in the flock; there is no central controller. Yet simulations show that if every bird follows only a simple *local* rule — stay diagonally behind the bird to its right, and fly at the same speed — the flock nevertheless forms a perfect, globally-ordered formation, regardless of how the birds start out or what obstacles are placed in their path. The global flocking pattern is an emergent, self-organised consequence of purely local, individual rules.

Self-organisation in the SOM. The SOM achieves the analogous effect for feature mapping: it is a competitive-learning network, so one neuron is chosen as the winner for each input, but — unlike plain k-means-style competitive learning — when the winning neuron’s weights are updated, the weights of its **neighbouring** neurons on the map are also updated (to a lesser extent), while neurons outside the neighbourhood are simply ignored (not repelled). This local neighbourhood-based update is the only “interaction” between neurons in the network, yet repeating it over many training inputs produces a globally ordered map in which the layout of neurons mirrors the layout of similarity relationships in the input space — this global result from local rules is exactly what is meant by self-organisation.



winner and its local neighbourhood (circled) are updated together

Figure 13: Only local neighbourhood interactions are used, yet training produces a globally-ordered map — self-organisation (after Fig. 14.9’s neighbourhood idea).

Concrete example. When a SOM is trained on random 2-D data drawn uniformly from $[-1, 1] \times [-1, 1]$, the network’s weights start completely disordered (no relationship between neuron position on the grid and the point in input space it represents). After only about 10 training iterations, the map becomes ordered: **neighbouring nodes on the grid map to nearby points in the input space**, purely as an emergent effect of the repeated local neighbourhood updates — no neuron was ever told directly what its neighbours’ inputs were, yet the ordering appears globally across the whole map. Similarly, when trained on the iris dataset, a 5×5 SOM organises itself so that the three iris classes end up occupying distinct, separate regions of the map, even though the network was given no information at all about the class labels during training.

Q18. Discuss the SOM algorithm in detail. How does neighbourhood adjustment affect learning?

Answer:

The **Self-Organising Feature Map** (SOM), proposed by Teuvo Kohonen (1988), is a competitive-learning network in which neurons are arranged with meaningful spatial relationships to one another (a 1-D line or, typically, a 2-D grid), and nearby neurons come to represent similar inputs — a property called **feature mapping** or **topology preservation**.

The SOM Algorithm.

Initialisation: choose the map size (number of neurons) and dimensionality d ; either choose random (but distinct) weight vectors, or initialise weights to increase along the first d principal components of the dataset (using PCA).

Learning — repeat: for each datapoint $\mathbf{x}$:

1. select the best-matching neuron n_b using minimum Euclidean distance:

$$n_b = \min_j \|\mathbf{x} - \mathbf{w}_j^T\|$$

2. update the weight vector of the best-matching (winning) node:

$$\mathbf{w}_j^T \leftarrow \mathbf{w}_j^T + \eta(t)(\mathbf{x} - \mathbf{w}_j^T)$$

where $\eta(t)$ is the (time-varying) learning rate;

3. update the weight vectors of *all other* neurons using:

$$\mathbf{w}_j^T \leftarrow \mathbf{w}_j^T + \eta_n(t) h(n_b, t) (\mathbf{x} - \mathbf{w}_j^T)$$

where $\eta_n(t)$ is the (separate, usually smaller) neighbourhood learning rate, and $h(n_b, t)$ is the **neighbourhood function**, equal to 1 if that neuron is within the current neighbourhood of the winner n_b , and 0 otherwise;

4. reduce the learning rates and shrink the neighbourhood, e.g., $\eta(t+1) = \alpha \eta(t)^{k/k_{\max}}$ where $0 \leq \alpha \leq 1$ controls how fast the size decreases, k is the current iteration count and $k_{\max}$ is when learning should stop (the same schedule is used for η , η_n and for $h(n_b, t)$).

until the map stops changing, or a maximum number of iterations is reached.

Usage: for each test point, select the best-matching neuron $n_b = \min_j \|\mathbf{x} - \mathbf{w}_j^T\|$.

The essential difference from plain competitive learning (k-means neural network) is step 3: not only does the winning neuron move towards the input, but its **neighbours** on the map also move towards the input, to a lesser extent, while neurons outside the neighbourhood are left untouched (ignored, not repelled). This is what is historically described using “Mexican Hat” lateral connections (a short-range excitatory, longer-range slightly inhibitory strength-vs-distance profile), but the SOM algorithm achieves the same effect more simply by modifying the weight-update rule directly rather than by using actual lateral connections.

How neighbourhood adjustment affects learning. The size of the neighbourhood (and the learning rates) is deliberately shrunk over the course of training, producing two distinct phases:

- **Ordering phase:** at the start of training, with random initial weights, the network is essentially unordered, so the neighbourhood is made *large*. This lets even distant neurons be pulled toward the same broad regions of input space together, quickly establishing the correct rough, large-scale ordering of the map.

- **Convergence phase:** once the rough ordering has been established, the neighbourhood is shrunk to be *small*, so that each neuron and only its immediate neighbours are fine-tuned to accurately represent their own local region of the input space, without disturbing the already-correct global order.

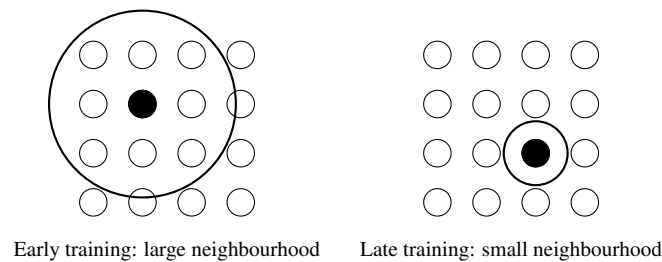


Figure 14: Neighbourhood size shrinks from large (ordering phase) to small (convergence phase) over training (after Fig. 14.9).

Practical implementation note. Because the neighbourhood’s actual extent changes throughout training, it is not implemented using fixed physical connections between nodes; instead, a distance matrix between all pairs of map nodes is precomputed, and at each iteration the neighbours of the winning node are simply those within the current (shrinking) neighbourhood radius in that distance matrix.

Example. On 2-D data drawn uniformly from $[-1, 1]^2$, a randomly-initialised SOM is completely disordered at the start; after only 10 iterations with a shrinking neighbourhood, the map becomes ordered so that neighbouring nodes represent nearby points in input space. Initialising the weights using PCA (so the map already lies along the two directions of greatest variance in the data) means the ordering phase is largely unnecessary from the start, so training can begin directly with a small neighbourhood — only 5 iterations are then needed to reach a similarly good map, but this shortcut requires *batch* training (all data available in advance), since the SOM is not designed for genuine online learning in that setting.